Growing Nexora Reports: Adding Many Apps

Nov 5, 2026 · @tanvik

Overview

One Django project can hold every report you build: broker commission, client activity, the dialer summary, calls after conversion and more. The trick is to put what every report shares into **one shared app, `core`**, so each new report app contains only what's specific to it. The project in `nexora_reports_django.zip` is now organised this way, with two report apps as examples. All 19 tests pass, and both reports were run in a browser.

App	Holds	Size
`core`	Ledger access, Excel styling, date and CSV helpers, the base report model, form checks, view helpers, the home page, the base layout, login	Shared, written once
`commission`	The broker commission rules, its two-sheet layout, its run model and pages	About 280 lines of Python, plus 3 templates
`activity`	Client trading activity: trading days, deposits, net withdrawals, traded or not, per-manager summary	About 235 lines of Python, plus 3 templates, written mostly by copying `commission`'s shape

What changed from the single-app version

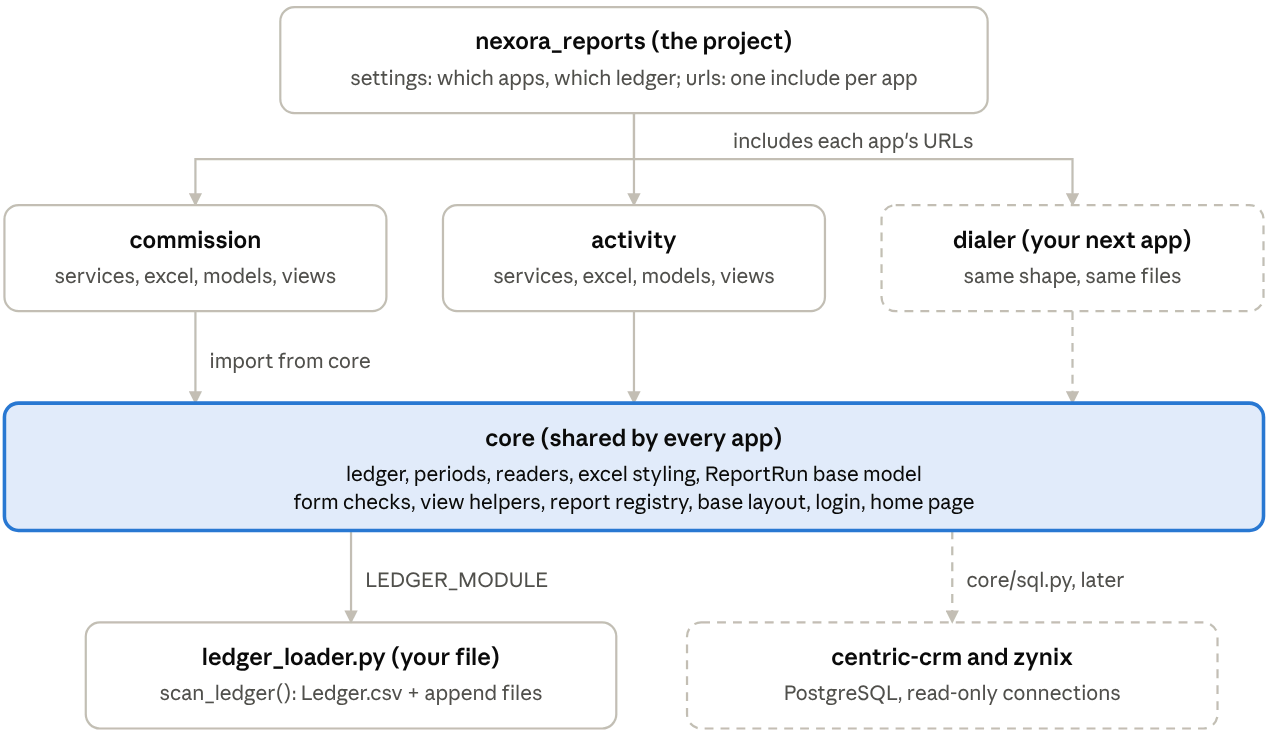
- `ledger.py`, `demo_ledger.py`, the base layout and the login page **moved to `core`**.
- The Excel styling became `core.excel.SheetSpec` + `workbook_bytes()`. `commission/excel.py` now only says which sheets and which bands.
- The fields every run has (`month`, `result_file`, `created_by`, `created_at`) moved into an **abstract model**, `core.models.ReportRun`.
- A **home page** lists every report app, and the menu builds itself from a registry. A new app appears there by registering itself; no template changes.
- A second app, **`activity`**, shows the recipe for adding one.

Three rules that keep many apps tidy

1. **Apps import from `core`, never from each other.** If two apps need the same thing, it belongs in `core`.
2. **The calculation lives in `services.py` with no Django imports.** It's plain pandas you can test on tiny tables and reuse in notebooks.
3. **Every app looks the same:** `services.py`, `excel.py`, `models.py`, `forms.py`, `views.py`, `urls.py`, `templates/<app>/`, `tests.py`, `sample_data/`. Once you know one app, you know them all.

How the pieces fit

Report apps depend only on core; only core knows the ledger



Report apps never import each other; anything two apps share moves into core. Dashed = not built yet.

nexora_reports · how the pieces depend on each other

Arrows point from the part that uses to the part that's used. Only `core` touches your `ledger_loader.py`, so a change to the loader never means editing every report.

What goes where

Before writing anything, ask: **would a second report need this too?** If yes, it goes in `core`. If it's one report's business rule, it goes in that report's app. If it configures the whole site, it goes in the project folder.

You're writing…	Put it in	Example
A business rule for one report	`<app>/services.py`	Brokerage on combined loss; "traded" means at least one BILL day
Which sheets, columns and bands one report's Excel has	`<app>/excel.py`	Commission's three coloured bands
How every Excel looks	`core/excel.py`	Gold headers, borders, money format, frozen headers
Reading the ledger	`core/ledger.py`	`get_ledger_slice(start, end, clients, types)`
Date and file helpers with no Django	`core/periods.py`, `core/readers.py`	`month_range`, `read_csv(file, required=...)`
Fields every saved run has	`core/models.py` (`ReportRun`)	`month`, `result_file`, `created_by`, `created_at`
Fields only one report has	`<app>/models.py`	Commission's `rate`; activity's `traded_count`
Checks used by several forms	`core/forms.py`	`clean_month_value`, `clean_csv_upload`
A page	`<app>/views.py` + `<app>/templates/<app>/`	`activity:run_detail`
The menu, header, styles, login page	`core/templates/`	`core/base.html`
Which apps exist, top-level URLs, database, secrets	`nexora_reports/settings.py`, `nexora_reports/urls.py`	`INSTALLED_APPS`, `LEDGER_MODULE`

The diagram above shows the import rule: report apps point at `core`, `core` points at your `ledger_loader.py`, and no arrow ever runs between two report apps. That keeps apps independent: you can delete, rename or rewrite one report without breaking another. When two apps start needing the same function, that's the signal to move it into `core`.

The core app, piece by piece

`core` has no report of its own. Every file in it exists so a report app doesn't have to write it again.

File	Gives every app	Django inside?
`ledger.py`	`get_ledger_slice(start, end, clients, types)`: only the rows a report needs	Yes (reads settings)
`demo_ledger.py`	Made-up data with the same interface as `ledger_loader.py`	No
`periods.py`	`month_range('2026-08')`, `previous_month()`	No
`readers.py`	`read_csv(file, required=[...], text_columns=[...])` with clear errors	No
`excel.py`	`SheetSpec`, `workbook_bytes([...])`, `col_numbers(df, names)`	No
`models.py`	`ReportRun`: the fields every saved run has	Yes
`forms.py`	`clean_month_value`, `clean_csv_upload`, `MONTH_WIDGET`	Yes
`web.py`	`download(file, name)`, `sheet_preview(file, sheet)`	Yes
`reports.py` + `context_processors.py`	The registry that builds the home page and menu	Yes
`templates/core/base.html`, `home.html`, `registration/login.html`	The look, the menu, the login page	Yes

Excel: describe the sheets, let core style them

```
@dataclass
class SheetSpec:
    name: str
    df: pd.DataFrame
    bands: list = field(default_factory=list)    # [(title, first col, last
col, colour)]; empty = no band row
    widths: list = field(default_factory=list)
    money_cols: tuple = ()
    center_cols: tuple = ()

def workbook_bytes(sheets: list[SheetSpec]) -> bytes: ...
```

A report app's `excel.py` becomes a short list of `SheetSpec`s. If you change the gold header colour in `core/excel.py`, every report changes with it. `col_numbers(df, ["Deposit", "Net Flow"])` finds column numbers by name, which matters when optional columns shift positions.

A `@dataclass` is a class whose `__init__` Python writes for you from the listed fields. It's a tidy way to bundle settings that travel together.

ReportRun: an abstract model

```
class ReportRun(models.Model):
    month = models.CharField(max_length=7)
    result_file = models.FileField(upload_to=result_path, blank=True)
    created_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                                   on_delete=models.SET_NULL,
related_name="+")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        abstract = True                  # no table of its own
        ordering = ["-created_at"]

class ActivityRun(ReportRun):            # in activity/models.py: gets all
four fields + ordering
    client_file = models.FileField(upload_to=input_path)
    traded_count = models.PositiveIntegerField(default=0)
```

- **`abstract = True`** means Django creates no table for `ReportRun`. Each child gets its own table containing the parent's fields plus its own.
- **`related_name="+"`** stops Django creating reverse links (`user.commissionrun_set`, `user.activityrun_set`, …). With many apps, those names would clash or clutter.
- **`upload_to=result_path`**, a function, files each app's results under `media/<app>/results/2026/11/`, so apps never mix files.

The registry: the home page and menu build themselves

```
# core/reports.py
@dataclass(frozen=True)
class Report:
    key: str
    title: str
    description: str
    url_name: str

REPORTS = []
def register(report): ...

# activity/apps.py
class ActivityConfig(AppConfig):
    name = "activity"
    def ready(self):                                    # runs once when
Django starts
        from core.reports import Report, register
        register(Report(key="activity", title="Client activity",
                        description="Trading days, deposits and withdrawals
per client...",
                        url_name="activity:run_list"))
```

A **context processor** (`core.context_processors.report_apps`, listed in `TEMPLATES` in settings) hands `REPORTS` to every template, so `base.html` loops over it to draw the menu. Once an app registers itself in `ready()` and its URLs are included in the project, it appears on the home page and in the menu with no template changes. (Register before including the URLs and every page breaks, because the menu can't build a link to a URL that doesn't exist yet.)

Pure modules stay pure

`periods.py`, `readers.py` and `excel.py` import no Django, so each app's `services.py` can import them and still be testable as plain pandas. Keep it that way: if a helper needs settings or requests, it belongs in `ledger.py`, `forms.py` or `web.py` instead.

The recipe: a new report app in ten steps

The order matters: the calculation first, tested in a notebook; the web parts last. If the pandas part is wrong, no amount of Django fixes it.

1. **Prove the calculation in a notebook** on real data, and write down the inputs (which upload, which ledger types, which month) and the output columns.
2. **Create the app:** `python manage.py startapp <name>`. Add `'<name>'` to `INSTALLED_APPS`, after `'core'`.
3. **`services.py`:** move the notebook code into functions with no Django imports: `read_<input>()`, one function per step, and one `build_<report>()` that returns the tables plus a few headline numbers.
4. **`excel.py`:** a `build_workbook()` that returns `workbook_bytes([SheetSpec(...), ...])`.
5. **`models.py`:** a run model that inherits `ReportRun` and adds the uploaded file and the headline numbers you want to list.
6. **`forms.py`:** a `ModelForm` for the run, using `clean_month_value` and `clean_csv_upload` from `core.forms`.
7. **`views.py`:** `run_list`, `run_create`, `run_detail`, `run_download`. Copy them from an existing app and change the model, form, service calls and file name.
8. **`urls.py`** with `app_name = "<name>"`, plus three templates in `templates/<name>/` that extend `core/base.html`. Include the app's URLs in `nexora_reports/urls.py`.
9. **Register it** in `apps.py` `ready()` with `register(Report(...))`, so it appears on the home page and in the menu.
10. **Migrate and test:** `python manage.py makemigrations <name>`, `python manage.py migrate`, tests for the services and one view test, then `python manage.py test`.

Commands, in order

```
python manage.py startapp dialer
# ...write the files...
python manage.py makemigrations dialer
python manage.py migrate
python manage.py test dialer          # just this app
python manage.py test                 # everything, before you call it done
python manage.py runserver
```

Worked example, part 1: activity's calculation and Excel

`activity` is your client-activity notebook as an app: upload a client list, pick a month, and get trading days, deposits, net withdrawals and traded or not per client, plus a summary per Manager and Assigned To.

services.py: the notebook, as functions

```
ACTIVITY_TYPES = ["BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW CANCELLED"]
OPTIONAL_COLUMNS = ["Name", "Manager", "Assigned To"]

def read_clients(file):
    df = read_csv(file, required=["Client ID"], text_columns=["Client ID"] +
OPTIONAL_COLUMNS)
    keep = ["Client ID"] + [c for c in OPTIONAL_COLUMNS if c in df.columns]
    return df[keep].dropna(subset=["Client ID"]).drop_duplicates("Client
ID").reset_index(drop=True)

def client_activity(led, clients):
    lt = led["LedgerType"].astype(str).str.strip().str.upper()
    debit, credit = led["Debit"].fillna(0), led["Credit"].fillna(0)
    per_client = (led
        .assign(Bill=lt.eq("BILL"),

TradeDate=pd.to_datetime(led["CreatedAt"]).dt.normalize().where(lt.eq("BILL")
),
                Deposit=credit.where(lt.eq("DEPOSIT"), 0),
                Withdraw=debit.where(lt.eq("WITHDRAW"), 0) -
credit.where(lt.eq("WITHDRAW CANCELLED"), 0))
        .groupby("ClientID")
        .agg(**{"Bills": ("Bill", "sum"), "Trading Days": ("TradeDate",
```

```
"nunique"),
                "Deposit": ("Deposit", "sum"), "Withdraw": ("Withdraw",
"sum")}))

    out = clients.merge(per_client, left_on="Client ID", right_index=True,
how="left")
    out[["Bills", "Trading Days"]] = out[["Bills", "Trading
Days"]].fillna(0).astype(int)
    out[["Deposit", "Withdraw"]] = out[["Deposit", "Withdraw"]].fillna(0.0)
    out["Net Flow"] = out["Deposit"] - out["Withdraw"]
    out["Status"] = np.where(out["Trading Days"] > 0, "Traded", "Not Traded")
    return out

def manager_summary(activity): ...                  # None when there's no
Manager column
def build_activity(led, clients): ...               # {"clients", "summary",
"headline"}
```

Every idea here is one you already use:

- **`.where(lt.eq(...), 0)`** builds each amount column, as in your deposit and withdrawal notebook.
- **`TradeDate` blank on non-BILL rows + `nunique`** counts trading days and ignores the blanks.
- **The `left` merge from the client list** keeps clients with no ledger rows; they get zeros and `Not Traded`.
- **Optional columns:** `Name`, `Manager` and `Assigned To` are kept only if the upload has them. `manager_summary` returns `None` when there's no Manager column, and the app simply skips that sheet.

excel.py: bands that follow the columns

```
def _client_bands(df):
    n_info = df.columns.get_loc("Bills")            # Client ID + whichever
optional columns came in
    return [("Client List", 1, n_info, "1F3864"),
            ("Trading", n_info + 1, n_info + 2, "375623"),
            ("Money (month)", n_info + 3, n_info + 5, "375623"),
            ("Result", n_info + 6, n_info + 6, "843C0C")]

def build_workbook(clients, summary=None) -> bytes:
    sheets = [SheetSpec("Clients", clients, bands=_client_bands(clients),
                        money_cols=col_numbers(clients, ["Deposit",
"Withdraw", "Net Flow"]), ...)]
    if summary is not None:
        sheets.append(SheetSpec("Manager Summary", summary, ...))
    return workbook_bytes(sheets)
```

Because the optional columns move everything after them, the bands and money columns are found by **name** (`get_loc`, `col_numbers`), not hard-coded positions.

Worked example, part 2: activity's Django parts

These files were copied from `commission` and changed in a few places. That's the payoff of every app having the same shape.

models.py and forms.py

```
class ActivityRun(ReportRun):                       # month, result_file,
created_by, created_at inherited
    client_file = models.FileField(upload_to=input_path)
    clients_count = models.PositiveIntegerField(default=0)
    traded_count = models.PositiveIntegerField(default=0)
    trading_days = models.PositiveIntegerField(default=0)
    net_flow = models.DecimalField(max_digits=16, decimal_places=2,
default=Decimal("0"))

    @property
    def traded_pct(self):                           # used in templates as
{{ run.traded_pct }}
        return round(self.traded_count / self.clients_count * 100, 1) if
self.clients_count else 0
```

```
class ActivityRunForm(forms.ModelForm):
    class Meta:
        model = ActivityRun
        fields = ["month", "client_file"]
        widgets = {"month": MONTH_WIDGET}

    def clean_month(self):
        return clean_month_value(self.cleaned_data["month"])

    def clean_client_file(self):
        return clean_csv_upload(self.cleaned_data["client_file"])
```

A `@property` is a value calculated on the fly instead of stored, handy for percentages that would go stale if saved.

views.py: what changed from commission

```
clients = services.read_clients(upload)                        # was
read_brokers
upload.seek(0)
start, end = month_range(run.month)
led, notes = get_ledger_slice(start, end, clients=clients["Client ID"],
                              types=services.ACTIVITY_TYPES)   # its own
ledger types
report = services.build_activity(led, clients)
content = excel.build_workbook(report["clients"], report["summary"])
...
run.result_file.save(f"Client_Activity_{run.month}.xlsx",
ContentFile(content))
return redirect("activity:run_detail", pk=run.pk)
```

The detail view shows two previews with `core.web.sheet_preview`. It returns `None` for a sheet that isn't in the file, so a client list without a Manager column simply has no summary card.

urls.py, templates, registration and project URLs

```
# activity/urls.py
app_name = "activity"
urlpatterns = [path("", views.run_list, name="run_list"), path("new/",
views.run_create, name="run_create"),
               path("<int:pk>/", views.run_detail, name="run_detail"),
               path("<int:pk>/download/", views.run_download,
name="run_download")]

# activity/apps.py
def ready(self):
    from core.reports import Report, register
    register(Report(key="activity", title="Client activity",
description="...", url_name="activity:run_list"))

# nexora_reports/urls.py
path("activity/", include("activity.urls")),
```

The three templates live in `activity/templates/activity/` and start with `{% extends "core/base.html" %}`. The extra `activity/` folder level matters: Django searches every app's `templates` folder, so two apps that both had a bare `run_list.html` would collide. With the folder, they're `activity/run_list.html` and `commission/run_list.html`.

Worked example, part 3: migrations, tests and checking

Migrations per app

```
python manage.py makemigrations activity      # writes
activity/migrations/0001_initial.py
python manage.py migrate                      # creates the
activity_activityrun table
```

Each app keeps its own `migrations/` folder and its own tables (`commission_commissionrun`, `activity_activityrun`). Changing one app's model never touches another's tables. `ReportRun` itself has no migration, because abstract models have no table.

Tests: a tiny ledger with every awkward case

```
self.led = pd.DataFrame({
    "ClientID":   ["A", "A", "A", "A", "A", "B"],
    "CreatedAt":  pd.to_datetime(["2026-08-02 09:00", "2026-08-02 15:00",
"2026-08-05 10:00",
                                  "2026-08-06 10:00", "2026-08-07 10:00",
"2026-08-09 10:00"]),
    "LedgerType": ["BILL", "BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW
CANCELLED", "DEPOSIT"],
    "Debit":      [10.0, 0.0, 0.0, 300.0, 0.0, 0.0],
    "Credit":     [0.0, 5.0, 1000.0, 0.0, 100.0, None],
})
self.clients = pd.DataFrame({"Client ID": ["A", "B", "C"], "Manager": ["m1",
"m1", None]})

def test_client_activity(self):
    out = services.client_activity(self.led, self.clients).set_index("Client
ID")
    self.assertEqual(out.loc["A", "Trading Days"], 1)        # two bills on
the same day
    self.assertEqual(out.loc["A", "Withdraw"], 200)          # 300 withdrawn,
100 cancelled
    self.assertEqual(out.loc["B", "Status"], "Not Traded")   # deposits only
    self.assertEqual(out.loc["C", "Deposit"], 0)             # no ledger rows
at all
```

Each row exists to catch one mistake: two bills on one day, a cancelled withdrawal, a blank Credit, a client with deposits but no trades, a client missing from the ledger, a client with no manager. The view tests upload the sample CSV, check the redirect and counts, and check that a file without `Client ID` is refused with a clear message.

One bug the tests found during this rebuild

The login page still extended the old `commission/base.html` after the layout moved to `core`. No test rendered the login page, so all 18 tests passed while the real site showed an error on the first page. Clicking through in a browser caught it, and `core` now has a test that renders the login page.

The lesson for many apps: **when you move a shared file, search every app for its old name** (`commission/base.html` here), and keep one test per shared page.

Before calling an app done

- [ ] `python manage.py check` reports no issues.
- [ ] `python manage.py test` passes for **every** app, not only the new one.
- [ ] Run the server, log in, and click through: the home page, the new app's list, form, detail and download.
- [ ] Run the same month in your notebook and compare one or two numbers with the app's Excel.

Conventions that keep many apps tidy

With two apps, anything works. With ten, only consistency does. These are the conventions both apps already follow.

Thing	Convention	Why
App names	Short, lowercase, one word: `commission`, `activity`, `dialer`	They become URL prefixes, table names and template folders
Run models	`<Name>Run(ReportRun)`	Every app's history works the same way
URL names	`app_name = "<app>"`; views named `run_list`, `run_create`, `run_detail`, `run_download`	`{% url 'dialer:run_list' %}` reads the same in every app
Templates	`<app>/templates/<app>/run_*.html`, extending `core/base.html`	No collisions; one look for every page
Business logic	`services.py`, no Django imports, one `build_<report>()` entry point	Testable; reusable from notebooks
Excel layout	`excel.py` with one `build_workbook()` returning bytes	The view never touches openpyxl
Ledger access	Only through `core.ledger.get_ledger_slice(...)` with the app's own `types`	One place that knows about `ledger_loader.py`
Sample data	`<app>/sample_data/` with a small, made-up input	Tests and demos run anywhere

Thing	Convention	Why
Tests	Services on a tiny table + at least one view test	A broken rule fails a test, not a payout
Output file names	`<Report>_<YYYY-MM>.xlsx`	Downloads sort by report and month

Using a notebook function from an app

Because `services.py` has no Django code, you can import it in a notebook to check a month by hand:

```
import sys
sys.path.insert(0, r"C:\...\nexora_reports")                        # the
project folder
from activity import services
from core.periods import month_range
from ledger_loader import scan_ledger
import polars as pl

start, end = month_range("2026-10")
led = scan_ledger().filter((pl.col("CreatedAt") >= start) &
(pl.col("CreatedAt") < end)).collect().to_pandas()
clients = services.read_clients(r"C:\...\trading-customers.csv")
report = services.build_activity(led, clients)
report["headline"]
```

The notebook and the web app now run the **same** rules. A fix in `services.py` fixes both.

When code starts repeating

With two apps, `views.py` and the three templates are still copied. That's deliberate: copying twice is cheaper than designing a general solution too early. By the third or fourth app, these are worth moving into `core`, in this order.

1. Generic list, detail and download views

`run_list`, `run_detail` and `run_download` differ between apps only in the model, the template and the file name. Django's class-based views can share them:

```
# core/views.py
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import DetailView, ListView

class RunListView(LoginRequiredMixin, ListView):
    paginate_by = 50                      # also gives page links for free

class RunDownloadView(LoginRequiredMixin, DetailView):
    filename = "Report_{month}.xlsx"
    def get(self, request, *args, **kwargs):
        run = self.get_object()
        return download(run.result_file,
self.filename.format(month=run.month))

# activity/urls.py
path("", RunListView.as_view(model=ActivityRun,
template_name="activity/run_list.html"), name="run_list"),
path("<int:pk>/download/", RunDownloadView.as_view(model=ActivityRun,

filename="Client_Activity_{month}.xlsx"), name="run_download"),
```

`run_create` stays in each app, because that's where the app's own form, services and headline numbers meet.

2. Shared template pieces

The headline numbers card and the preview card look the same everywhere. Move them to `core/templates/core/_stats.html` and `_preview.html`, and include them:

```
{% include "core/_preview.html" with title="First 15 clients" table=clients
%}
```

3. A shared "generate" step

Every `run_create` ends the same way: save the run, save the Excel, show the notes, redirect. One helper in `core/web.py` can do all of it:

```
def finish_run(request, run, content, filename, notes, detail_url_name):
    run.save()
    run.result_file.save(filename, ContentFile(content))
    for note in notes:
        messages.warning(request, note)
    messages.success(request, f"{filename} is ready.")
    return redirect(detail_url_name, pk=run.pk)
```

When to do it: when you're about to copy the same lines into a third app. Move them, run every app's tests, and only then write the third app.

Who can see which report

Right now any logged-in user can open every report. With commission figures in one app and call data in another, you'll want Finance to see commission and team leads to see activity, but not each other's.

Django already made the permissions

For every model, `migrate` creates four permissions: `add`, `change`, `delete` and `view`. So `commission.view_commissionrun` and `activity.view_activityrun` already exist.

1. Groups in the admin

In `/admin/` → **Groups**, create *Finance* with the `commission | commission run` permissions, and *Team Leads* with the `activity | activity run` ones. Then add each user to a group.

2. Check the permission in the views

```
from django.contrib.auth.decorators import permission_required

@permission_required("activity.view_activityrun", raise_exception=True)
def run_list(request): ...

@permission_required("activity.add_activityrun", raise_exception=True)
def run_create(request): ...
```

`raise_exception=True` shows a "403 Forbidden" page instead of sending a logged-in user back to the login screen.

3. Hide what they can't open

Add a `permission` field to `Report` (for example `"activity.view_activityrun"`), and filter the registry for the current user in the context processor:

```
def report_apps(request):
    visible = [r for r in REPORTS if request.user.has_perm(r.permission)]
    return {"report_apps": visible}
```

The home page and menu then show each person only their reports. Superusers pass every `has_perm` check, so you still see everything.

Settings as the project grows

One `settings.py` is fine on your own machine. Once the site runs on a server for the team, you need two versions of a few settings: the server must never run with `DEBUG = True` or a secret key that's in the code.

Split into base, dev and prod

```
nexora_reports/settings/
    __init__.py
    base.py        # everything shared: INSTALLED_APPS, TEMPLATES, MEDIA,
LEDGER_* defaults
    dev.py         # from .base import *   DEBUG = True, SQLite, demo ledger
    prod.py        # from .base import *   DEBUG = False, PostgreSQL, real
ledger, secrets from env
```

```
# prod.py
from .base import *
import os

DEBUG = False
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]                    # fails loudly
if missing
ALLOWED_HOSTS = ["reports.nexora.local"]
DATABASES = {"default": {"ENGINE": "django.db.backends.postgresql", "NAME":
"nexora_reports",
                         "USER": os.environ["DB_USER"], "PASSWORD":
os.environ["DB_PASSWORD"],
                         "HOST": "localhost", "PORT": "5432"}}
LEDGER_MODULE = "ledger_loader"
LEDGER_MODULE_PATH = r"C:\Users\Administrator\Videos\Office\migration"
```

One catch when you move the file: `BASE_DIR` in `base.py` needs one more `.parent` (`Path(__file__).resolve().parent.parent.parent`), because the file now sits a folder deeper. Choose the file with an environment variable (`setx DJANGO_SETTINGS_MODULE nexora_reports.settings.prod` on the server), the same way as `UNO_API_TOKEN`.

Settings that belong to one app

Give app settings the app's name as a prefix, read them with a default, and they never clash:

```
# settings/base.py
COMMISSION_DEFAULT_RATE = 0.05
ACTIVITY_TYPES = ["BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW CANCELLED"]

# activity/views.py
types = getattr(settings, "ACTIVITY_TYPES", services.ACTIVITY_TYPES)
```

Secrets never go in code

Database passwords, the Uno API token and the Django secret key come from environment variables. If a secret was ever committed or pasted somewhere, change it.

Your next apps

Most of the reports you've built in notebooks and Metabase over the last weeks fit this shape. The question for each is where its data comes from, because that decides which new piece `core` needs.

App	Input	Data from	New core piece it needs
`dialer`	Stringee and Uno CDR exports, the team list	Uploaded files only	Several files in one upload
`conversion_calls`	Client IDs (CSV) and a month	`centric-crm` PostgreSQL: calls, leads, team	A read-only database connection + `run_sql()`
`lead_sources`	None (all depositors)	`centric-crm` PostgreSQL	The same `run_sql()`
`campaign`	Client list, campaign date and amount	Ledger (DEPOSIT, BILL)	Nothing new; could be an option in `activity`
`zero_balance`	Client list	Ledger (latest balance, last BILL)	Nothing new
`unreachable_leads`	None	`centric-crm` PostgreSQL	`run_sql()`
`shared_ips`	None	`zynix` PostgreSQL (`tdx_activity`)	A second connection

The core piece most of them need: SQL

Three of these read from `centric-crm` instead of the ledger. One shared helper covers them all:

```
# settings: read-only connections next to the default database
DATABASES["crm"] = {"ENGINE": "django.db.backends.postgresql", "NAME":
"centric-crm",
                    "USER": os.environ["CRM_USER"], "PASSWORD":
os.environ["CRM_PASSWORD"],
                    "HOST": "...", "PORT": "5432",
                    "TEST": {"MIRROR": "default"}}      # tests never create
or touch a CRM database

# core/sql.py
import pandas as pd
from django.db import connections

def run_sql(alias: str, query: str, params=None) -> pd.DataFrame:
    """Run a read-only query on a named database and return a DataFrame."""
    with connections[alias].cursor() as cur:
        cur.execute(query, params or [])
        columns = [c[0] for c in cur.description]
        return pd.DataFrame(cur.fetchall(), columns=columns)
```

Two cautions when moving a Metabase query into an app:

- **Metabase-only syntax doesn't exist in plain SQL.** `{{snippet: portal_id}}` and `{{#3268-team-main}}` have to become real values or CTEs.
- **Pass values as parameters, never pasted into the SQL text.** `WHERE zynixx_client_id = ANY(%s)` with `params=[list_of_ids]` keeps a stray quote in an uploaded file from breaking, or rewriting, the query.

Use a database user with **read-only** rights for these connections, so no report can ever change CRM data. `"TEST": {"MIRROR": "default"}` matters too: without it, `python manage.py test` tries to create a test copy of the CRM database on the CRM server.

A suggested order

1. **`dialer`**: it saves you the most time every day, and it needs nothing new except a multi-file upload.
2. **`core/sql.py`** + **`conversion_calls`**: one new core piece unlocks three apps.
3. **`campaign`** and **`zero_balance`**: small, and good practice at the recipe.

Practice: build the dialer app yourself

Your `dialer_report.py` already has the pandas part. Turning it into the third app uses everything above, plus two new Django ideas: uploading several files in one field, and a model linked to many files.

The two new ideas

Several files in one field. Django's documented pattern, which belongs in `core/forms.py` so every app can use it:

```
class MultipleFileInput(forms.ClearableFileInput):
    allow_multiple_selected = True

class MultipleFileField(forms.FileField):
    def __init__(self, *args, **kwargs):
        kwargs.setdefault("widget", MultipleFileInput())
        super().__init__(*args, **kwargs)

    def clean(self, data, initial=None):
        single = super().clean
        if isinstance(data, (list, tuple)):
            return [single(d, initial) for d in data]
        return [single(data, initial)]
```

One run, many uploaded files, through a second model with a `ForeignKey`:

```
class DialerRun(ReportRun):
    start_date = models.DateField()
    end_date = models.DateField()
    calls_count = models.PositiveIntegerField(default=0)

class DialerInput(models.Model):
    run = models.ForeignKey(DialerRun, on_delete=models.CASCADE,
related_name="inputs")
    source = models.CharField(max_length=20)          # "stringee", "uno" or
"team"
    file = models.FileField(upload_to=input_path)
```

`on_delete=CASCADE` deletes a run's input rows with the run. `run.inputs.all()` lists them on the detail page.

When `ReportRun` doesn't quite fit: the dialer report covers a date range, not a calendar month. Keep `month` (set it from `start_date`, so lists still group by month) and add `start_date` and `end_date`. A shared base model is a starting point, not a cage.

Steps

- [ ] Add `MultipleFileInput` and `MultipleFileField` to `core/forms.py`, with a test that a list of two files comes back as two files.
- [ ] `python manage.py startapp dialer`, then add it to `INSTALLED_APPS`.
- [ ] `dialer/services.py`: move the cells of `dialer_report.py` into `read_stringee(files)`, `read_uno(files)`, `read_team(file)` and `build_dialer(...)`, returning Summary, Hourly, Dialer and Not_Found_Users. Keep the settings block as module constants.
- [ ] `dialer/excel.py`: four `SheetSpec`s, with durations formatted as `[h]:mm:ss` (seconds ÷ 86,400).
- [ ] `DialerRun` and `DialerInput`, then `makemigrations dialer` and `migrate`.
- [ ] A form with `start_date`, `end_date`, `stringee_files`, `uno_files` and `team_file`.
- [ ] `run_create`: save each upload as a `DialerInput`, build the report, save the Excel, redirect.
- [ ] URLs, three templates, registration in `apps.py`, project URL include.
- [ ] Tests: services on two tiny CDR files (one Stringee, one Uno, with a gap over 60 seconds and a duplicate call), and one view test.
- [ ] Run `python manage.py test` for every app, click through in the browser, and compare one day's Summary with the notebook's output.